Where Do OpenMP Threads Help? A Stage-by-Stage Performance Analysis of a Multi-Scale Frangi Pipeline for Retinal Vessel Segmentation

Devik Nangia, Snehal Tyagi, Srijan Bera, and Prathamesh Patil

Manipal Institute of Technology, Manipal, India

Abstract. Diabetic retinopathy screening depends on fast and reliable segmentation of retinal vessels, and the multi-scale Frangi vesselness filter is a training-free classical method whose cost grows with image resolution and with the number of scales. We implement a complete pipeline (green-channel extraction, CLAHE, multi-scale Frangi filtering, thresholding and morphological clean-up) in C, parallelise every stage with OpenMP, and compare it stage by stage with a pragma-free serial build of the same code on a 2-core, 4-thread cloud CPU. Serial and OpenMP outputs are bit-identical on all 20 DRIVE test images for every thread count and design variant. With parameters tuned on the DRIVE training images only, the pipeline reaches a mean Dice of 0.713 on the DRIVE test set and 0.654 on STARE. The Gaussian-derivative passes take 72% of the 108 ms serial time. Four threads speed the whole pipeline up by 2.08× (19.2 images per second), but placement matters: two threads on the two hyperthreads of one core give 1.08×, against 1.98× when they are spread over both cores, and Hyper-Threading adds only 7%. Owner-computes tile histograms are 1.68× faster than atomic histograms in CLAHE, and running the four Frangi scales concurrently is 1.17× slower than parallelising inside each scale because of load imbalance. The 4-thread speedup stays between 2.13 and 2.31 from 0.33 to 21.1 megapixels. We report per-stage Karp–Flatt serial fractions and throughput, and place the accuracy beside published lightweight deep models.

Keywords: Parallel computing · OpenMP · Multicore · Hyper-Threading · Frangi filter · CLAHE · Retinal vessel segmentation · Diabetic retinopathy

1 Introduction

Diabetic retinopathy damages the small blood vessels of the retina and is a leading preventable cause of vision loss. It is diagnosed from colour fundus photographs, and the first computational step in most automated screening systems is segmenting the retinal vessels [5]. India alone had an estimated 89.8 million adults with diabetes in 2024 [9]. Screening camps capture fundus images in bulk, often on modest hardware and with few specialists available to read them, so throughput (images per second) is a practical requirement.

The multi-scale Frangi vesselness filter [4] is a widely used classical method for this step. It needs no training data, behaves deterministically and can be written in plain C.

Its cost grows with image resolution and with the number of scales, because every pixel needs several Gaussian-derivative convolutions followed by an eigenvalue computation. A complete screening pipeline adds contrast enhancement before the filter and binary clean-up after it, and these stages stress a processor in different ways: convolution is memory-bound, the eigenvalue step is arithmetic-heavy, CLAHE builds histograms, and morphology moves single bytes.

Multicore CPUs are the hardware every screening site already has. A developer choosing how to parallelise such a pipeline needs to know, stage by stage, where OpenMP threads help, where they stop helping, and why. Published parallel retinal work has used other algorithms or isolated single stages [1,2], and the closest cross-platform study covers a single enhancement filter [17].

This paper implements the full pipeline (green-channel extraction, CLAHE, multi-scale Frangi filtering, thresholding and morphological clean-up) in C, parallelises every stage with OpenMP, and analyses its performance stage by stage against a pragma-free serial build of the same code. Accuracy is measured on the DRIVE benchmark [18], with STARE [8] as a cross-dataset check, using parameters tuned on the DRIVE training images only. The contributions are:

- a documented serial/OpenMP C implementation whose outputs are bit-identical across all thread counts and design variants, verified on every test image;
- a stage-by-stage measurement of time, speedup, efficiency and Karp–Flatt serial fraction on a 2-core/4-thread cloud CPU, including the effect of Hyper-Threading and thread placement;
- an ablation of three alternative OpenMP designs (dynamic tile scheduling, atomic histograms and scale-level parallelism) and a resolution-scaling study from 0.33 to 21.1 megapixels;
- segmentation accuracy on DRIVE and STARE, set beside the accuracy of published lightweight deep models.

The study tests four hypotheses. H1 and H4 are taken unchanged from the project proposal. The proposal's two GPU hypotheses fall outside this OpenMP-only study and are replaced by H2 and H3, which were written after a short pilot run (two images, three repetitions) used to debug the harness; the pilot data are not reported. **H1**: OpenMP speedup grows sub-linearly with thread count and is limited by the number of physical cores and by memory bandwidth. **H2**: OpenMP speedup is stable or grows with image size, because the fixed costs of creating parallel regions shrink relative to the work. **H3**: the best decomposition differs by stage: owner-computes tile histograms beat atomic histograms in CLAHE, and parallelism inside each scale beats running the scales concurrently in the Frangi stage, because the four scales have unequal cost. **H4**: the classical pipeline does not match the best lightweight deep models in Dice (for example, 0.8070 on DRIVE for LightVesselNet [16]); its value is that it is training-free and its parallel behaviour can be explained.

Section 2 reviews related work, Section 3 describes the pipeline, the OpenMP design and the measurement protocol, Section 4 reports the results and Section 5 discusses them against the hypotheses.

2 Related Work

2.1 Surveys and accuracy-centred work

Fraz et al. [5] categorise classical retinal vessel extraction methods, including Hessian-based vesselness, and compare them on DRIVE [18] and STARE [8]. Later reviews concentrate on deep networks: Chen et al. [3] survey architectures and open problems, Sule [19] organises 110 papers from 2016–2021 by the technique used to improve accuracy, and Gojić et al. [7] summarise recent methods with their reported metrics. Execution time on commodity hardware is not an organising axis in any of these surveys.

2.2 Classical vesselness methods

The Frangi filter [4] scores each pixel by how tube-like its Hessian eigenvalue structure is, over several Gaussian scales. Mahapatra et al. [11] keep this interpretable core, tune its parameters with particle-swarm optimisation and follow it with fuzzy clustering. The method is unsupervised, but the optimisation and the clustering add computation, and parallel execution is not studied.

2.3 Lightweight deep models

Galdran et al. [6] show that minimalistic U-Nets approach state-of-the-art accuracy and that cross-dataset generalisation remains weak. Deployment-oriented networks report model size instead of measured time: LVS-Net [13] (0.71 M parameters, 29.60 GFLOPs, DRIVE Dice 0.8644), LFA-Net [14] (0.11 M parameters, 4.46 GFLOPs, DRIVE Dice 0.8328) and LightVesselNet [16] (75 K parameters, Dice 0.8070 on DRIVE and 0.8072 on STARE). These models need annotated training data, trained weights and a deep-learning runtime.

2.4 Parallel and accelerated vessel analysis

Argüello et al. [1] map a filtering-plus-contour-tracing method to CUDA and segment a DRIVE image in 14 ms on a GTX 680. Ben Sayadia et al. [2] accelerate the Hessian eigenvalue step on a GPU and report a 27× speedup over a multithreaded CPU version, applied to thick-vessel segmentation. Nahar et al. [15] move Hessian-based filtering of 3-D mitochondrial images to the GPU (up to 11× and 30×) and note that small inputs under-use the device. Song et al. [17] compare serial, OpenMP, CUDA and OpenCL versions of unsharp masking over image sizes from 525^2 to $16{,}364 \times 8{,}182$ and report OpenMP speedups that track the core count. That study is the closest methodological template for ours, but it covers a single enhancement filter and no vessel analysis.

2.5 Gap

We found no study that parallelises a complete CLAHE–Frangi–morphology pipeline on a multicore CPU and profiles every stage. Prior parallel retinal work either uses a different algorithm [1] or a single stage [2]. Generic comparisons [17] cover one filter

whose behaviour may not transfer, because the Frangi pipeline mixes a memory-bound convolution stage, an arithmetic-heavy eigenvalue stage, a histogram stage and byte-wide morphology.

3 Method

The serial and the OpenMP builds are compiled from one C source file set. The serial build omits `-fopenmp`, so every pragma is ignored and the same loops run on one thread. Any difference in time is therefore due to parallel execution, not to a different algorithm.

3.1 Pipeline

Stage 1, green channel. $G(x, y)$ is taken from the RGB fundus image, where vessels have the highest contrast. DRIVE images are LZW-compressed TIFF files, which we decode with our own C reader (verified pixel-exact against Pillow); STARE images are PPM files.

Stage 2, CLAHE [22]. The image is split into an 8×8 grid of tiles of $\lceil w/8 \rceil \times \lceil h/8 \rceil$ pixels. Each tile's 256-bin histogram is clipped at $\kappa N_t/256$ counts (N_t tile pixels, κ the clip factor), the excess is spread uniformly over the bins, and the cumulative histogram gives a lookup table. Each output pixel is a bilinear blend of the four nearest tile tables.

Stage 3, multi-scale Frangi filter [4]. With I the CLAHE output scaled to $[0, 1]$, the second derivatives at scale σ are computed by separable 1-D convolutions with sampled Gaussian kernels of radius $r = \lceil 3\sigma \rceil$: $I_{xx} = g_\sigma^{(y)} * (g_\sigma^{\prime\prime(x)} * I)$, $I_{yy} = g_\sigma^{\prime\prime(y)} * (g_\sigma^{(x)} * I)$ and $I_{xy} = g_\sigma^{\prime(y)} * (g_\sigma^{\prime(x)} * I)$, with replicated borders. The kernels are normalised so that $\sum g = 1$, $\sum k\, g'(k) = -1$, $\sum g'' = 0$ and $\sum \frac{1}{2} k^2 g''(k) = 1$. One row sweep produces the three row-filtered images and one column sweep the three derivatives, i.e. six 1-D passes per scale. With $a = \sigma^2 I_{xx}$, $b = \sigma^2 I_{xy}$ and $d = \sigma^2 I_{yy}$, the scale-normalised Hessian has eigenvalues

$$\lambda_{1,2} = \tfrac{1}{2}(a + d) \mp \sqrt{\tfrac{1}{4}(a - d)^2 + b^2}, \tag{1}$$

relabelled so that $|\lambda_1| \le |\lambda_2|$. Vessels are dark on a bright background, so the response is non-zero only when $\lambda_2 > 0$. With $R_B = |\lambda_1|/\lambda_2$ and $S^2 = \lambda_1^2 + \lambda_2^2$,

$$\mathcal{V}_\sigma = \exp\!\left(-\frac{R_B^2}{2\beta^2}\right)\left[1 - \exp\!\left(-\frac{S^2}{2c^2}\right)\right], \tag{2}$$

and $V(x, y) = \max_{\sigma \in \{1,2,3,4\}} \mathcal{V}_\sigma(x, y)$.

Stage 4, threshold and morphology. V is thresholded at a global value τ and multiplied by the field-of-view (FOV) mask eroded by e pixels, which removes the bright ring that the filter produces on the FOV border. A 3×3 binary opening (erosion then dilation) removes speckle and a 3×3 closing bridges one-pixel gaps.

Stage 5, evaluation. True and false positives and negatives are counted inside the (uneroded) FOV and converted to Dice $= 2TP/(2TP + FP + FN)$, sensitivity, specificity and accuracy.

Table 1. Parallel decomposition by stage.

Stage	OpenMP decomposition
Green channel	`parallel for` over pixels
CLAHE	`parallel for` over the 64 tiles, each thread owning its tile histogram and lookup table; `parallel for` over rows for the bilinear blend
Gaussian derivatives	row sweep: `parallel for` over rows with a thread-private padded row buffer; column sweep: `parallel for` over output rows, inner loop over x vectorised
Eigen + vesselness	`parallel for` over pixels, fused with the running maximum over scales
Threshold	`parallel for` over pixels, fused with the FOV mask
Morphology	four 3×3 passes, each a `parallel for` over rows
Metrics	`reduction(+:TP,FP,TN,FN)`

3.2 Parameter selection

All parameters are chosen on the 20 DRIVE training images only, by maximising the mean per-image Dice over a grid: $\kappa \in \{1, 2, 3, 4\}$, $\beta \in \{0.5, 1\}$, $c \in \{0.005, 0.01, 0.02, 0.04, 0.08, 0.16\}$, $e \in \{4, 8, 12\}$ and 40 thresholds spaced logarithmically in $[10^{-3}, 0.9]$ (5,760 settings). The selected setting is then fixed and applied unchanged to the 20 DRIVE test images, scored against the first observer's manual segmentation, and to the 20 STARE images, scored against Hoover's labels [8]. STARE has no FOV masks, so the FOV is the set of pixels whose maximum channel value exceeds 35, cleaned by a 3×3 opening.

3.3 OpenMP design

Table 1 lists the parallel decomposition of each stage. All loops use `schedule(static)` unless stated, and each output element is computed by exactly one thread, so floating-point summation order is the same as in the serial build. Three alternative designs are kept as ablations: a dynamic schedule for the CLAHE tile loop, a pixel-parallel CLAHE histogram with `#pragma omp atomic` increments, and a Frangi decomposition that runs the four scales concurrently with one thread each instead of parallelising inside each scale.

3.4 Correctness checks

For every test image the serial build writes the CLAHE output, the vesselness map V and the final mask, and every OpenMP configuration (each design variant at 1, 2, 4 and 8 threads) writes the same files. We report the fraction of identical CLAHE bytes, identical mask pixels and bit-identical V values, and the maximum $|\Delta V|$. The comparison is repeated for builds with floating-point contraction disabled (`-ffp-contract=off`). As an external check, V from the serial build is compared with `skimage.filters.frangi` [20] applied to the same CLAHE output with the same σ, β and c; the two

implementations normalise derivatives differently, so we report correlation rather than equality.

3.5 Platform and measurement protocol

All timed runs use one Kaggle CPU notebook: an Intel Xeon at 2.20 GHz exposing 4 logical CPUs (2 physical cores with 2-way Hyper-Threading [12]), 32 KiB L1d and 256 KiB L2 per core, a 55 MiB L3 and 31 GiB RAM, with a cgroup quota of 4 CPUs, running Linux 6.18 with GCC 13.3 (`-O3 -march=native`, OpenMP 4.5). Nothing is timed on the authors' laptops. Thread counts are 1, 2, 4 and 8 (the last oversubscribes the 4 logical CPUs); threads are bound with `OMP_PROC_BIND=close` and `OMP_PLACES=threads` unless stated. Each configuration runs 5 untimed warm-up iterations and 30 timed iterations per image over all 20 DRIVE test images. Time is measured with `omp_get_wtime` (OpenMP build) or the monotonic clock (serial build) around each stage; file I/O and image decoding are excluded. Speedup is $S(p) = T_{\text{serial}}/T(p)$ with T_{serial} from the pragma-free build, efficiency is $E = S/p$ and the Karp–Flatt serial fraction [10] is $e = (1/S - 1/p)/(1 - 1/p)$. Throughput is images per second for one image at a time.

For resolution scaling, three test images are bilinearly resampled to 2×, 4× and 8× linear size (0.33 to 21.1 megapixels) before timing, with the FOV mask resampled by nearest neighbour and the erosion radius scaled accordingly; the number of timed iterations is reduced at large sizes (10, 10, 5 and 3). The three design ablations are timed on the first five test images with the same protocol. A thread-placement experiment repeats the sweep on the same five images with one thread per physical core first (`OMP_PLACES=cores`, `OMP_PROC_BIND=spread`) and with binding disabled. STARE is timed on all 20 images with 2 warm-up and 10 timed iterations.

4 Results

The full run (tuning, accuracy, verification and all timing experiments) took 28.5 minutes on the Kaggle VM. `lscpu -e` confirms the topology: logical CPUs 0 and 2 share physical core 0, and CPUs 1 and 3 share core 1 (CPU family 6, model 79).

4.1 Correctness

Table 2 lists the four verification checks. The first is the one that matters for the parallel study: across 33 OpenMP configurations (four designs at $p = 1, 2, 4, 8$, each in a build with and without floating-point contraction, plus STARE at $p = 4$) and 6.6 million pixels per configuration, every CLAHE byte, every float of the vesselness map V and every mask pixel equals the serial result. This holds by construction, because each output element is computed by one thread in the serial summation order, so OpenMP changes when values are computed but never how. The second check shows why that matters: compiler contraction alone does change the arithmetic, leaving only 48.6% of V values bit-identical between the serial FMA and no-FMA builds ($\max |\Delta V| = 2.2 \times 10^{-3}$), yet no mask pixel changes. A tolerance-based check would therefore have been needed if the parallel code had reordered any sum. The last two checks validate the implementation

Table 2. Verification summary (DRIVE test images unless stated).

Check	Scope	Result
Serial vs OpenMP: 4 designs × $p \in \{1, 2, 4, 8\} \times 2$ builds, plus STARE	33 configs, 20 images, CLAHE, V, mask	all bit-identical
Serial FMA vs serial no-FMA build	20 images, V, mask	48.6% of V bit-identical, $\max \lvert\Delta V\rvert = 2.2 \times 10^{-3}$; masks identical
C TIFF decoder vs Pillow	3 images, green channel	identical
C Frangi vs `skimage frangi`	4 images, V in FOV	Pearson 0.785 (0.755–0.811), Spearman 0.877

Table 3. Segmentation accuracy inside the FOV (mean over 20 images; Dice ± s.d.). Deep-model figures are as reported by their authors.

Set	Dice	Sens.	Spec.	Acc.
DRIVE test	0.7126 ± 0.0304	0.7020	0.9615	0.9283
DRIVE train (tuning)	0.6871 ± 0.0734	0.7081	0.9474	0.9170
STARE (cross-dataset)	0.6540 ± 0.0731	0.7906	0.9284	0.9151
LightVesselNet [16], DRIVE	0.8070	0.8189	–	–
LightVesselNet [16], STARE	0.8072	0.8499	–	–
LFA-Net [14], DRIVE	0.8328	–	–	–
LVS-Net [13], DRIVE	0.8644	–	–	–

itself. Our TIFF decoder matches Pillow exactly. Our filter correlates with `skimage.filters.frangi` on the same input (Pearson 0.785, Spearman 0.877) but does not match it, because the two normalise the derivative kernels differently.

4.2 Segmentation accuracy

The grid search selected $\kappa = 4$, $\beta = 0.5$, $c = 0.04$, $e = 4$ and $\tau = 0.316$, with a mean training Dice of 0.6871. Table 3 gives the accuracy with these settings fixed. On the DRIVE test set the mean Dice is 0.7126 (range 0.6497 for image 15 to 0.7739 for image 02). On STARE, with the DRIVE parameters unchanged, it is 0.6540. Image im0003 scores lowest (0.4322): it is covered with many small bright lesions, and the darker background between neighbouring lesions forms short dark ridges that the filter accepts as vessels.

Figure 1 shows two test images. Most missed pixels (blue) are thin capillaries one or two pixels wide and the outer edges of wide vessels; false positives (orange) cluster around the optic disc and appear as short isolated fragments in the background.

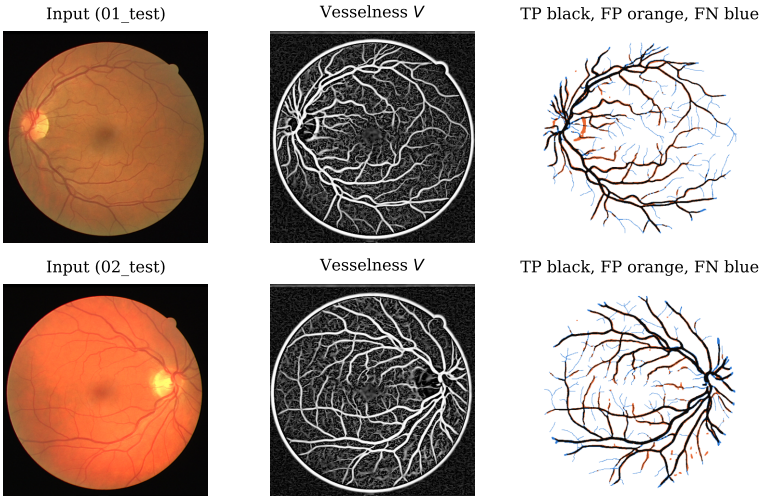


Fig. 1. DRIVE test images 01 and 02: input, vesselness V, and the final mask compared with the manual segmentation (true positives black, false positives orange, false negatives blue).

Table 4. Serial time per DRIVE test image by stage (20 images × 30 runs).

Stage	Mean (ms)	S.d.	Median	Share (%)
Green channel	0.12	0.01	0.11	0.1
CLAHE	5.81	0.15	5.78	5.4
Gaussian derivatives	77.97	2.30	77.46	72.1
Eigen + vesselness	23.93	0.51	23.84	22.1
Threshold	0.16	0.01	0.16	0.1
Morphology	0.21	0.02	0.21	0.2
Total	108.20	2.79	107.63	100

4.3 Serial profile

Table 4 breaks the serial time per image into stages. The Gaussian-derivative passes take 72.1% of the 108.2 ms and the eigenvalue/vesselness stage 22.1%, so the Frangi filter accounts for 94.2% of the time. CLAHE takes 5.4%; green-channel extraction, thresholding and morphology together take under 0.5%. Run-to-run variation is small: the coefficient of variation of the total is 2.6%.

4.4 Thread scaling

Table 5 and Figs. 2–4 give the whole-pipeline and per-stage scaling with the default close binding. The OpenMP build at $p = 1$ runs as fast as the serial build (speedup 1.00), so the pragmas add no single-thread overhead. With two close-bound threads the speedup is only 1.07. At $p = 4$ it rises to 2.08 (19.2 images per second), and oversubscribing to 8

Table 5. Whole-pipeline scaling on the 20 DRIVE test images (`OMP_PROC_BIND=close`, `OMP_PLACES=threads`).

p	Time (ms)	S	E	e	Images/s
serial	108.2	1.00	–	–	9.24
1	107.7	1.00	1.00	–	9.29
2	100.7	1.07	0.54	0.861	9.93
4	52.0	2.08	0.52	0.307	19.24
8	53.0	2.04	0.25	0.417	18.85

Table 6. Per-stage speedup over the serial build (close binding).

Stage	$p = 1$	$p = 2$	$p = 4$	$p = 8$
Green channel	0.98	1.18	2.15	1.08
CLAHE	0.99	1.13	2.18	2.14
Gaussian derivatives	1.00	1.02	1.99	1.96
Eigen + vesselness	1.02	1.26	2.43	2.39
Threshold	0.98	1.10	1.69	1.01
Morphology	0.93	0.94	1.42	0.59
Total	1.00	1.07	2.08	2.04

threads lowers it slightly to 2.04. The Karp–Flatt fraction is 0.86 at $p = 2$, falls to 0.31 at $p = 4$ and rises again to 0.42 at $p = 8$.

Table 6 shows that the stages scale differently. The eigen/vesselness stage scales best (2.43 at $p = 4$), followed by CLAHE (2.18) and the green channel (2.15); the dominant Gaussian-derivative stage reaches 1.99. Morphology reaches only 1.42 at $p = 4$ and slows down at $p = 8$ (0.59), where its Karp–Flatt fraction exceeds 1. Thresholding and the green channel also lose most of their gain at $p = 8$ (Fig. 4).

4.5 Thread placement

Table 7 and Fig. 5 repeat the sweep on five test images with three placements. With close binding, libgomp lists the hardware threads core by core, so two threads run on the two hyperthreads of core 0 and reach only 1.08×. Placing one thread per physical core (`spread`, `cores`) gives 1.98× at $p = 2$, an efficiency of 0.99; leaving the threads unbound gives the same result because the Linux scheduler also spreads them. At $p = 4$ every logical CPU is busy and the placements differ by at most 4%. Going from two threads on two cores (54.9 ms) to four threads on two cores (51.2 ms with close binding) gains 7%, which is the whole Hyper-Threading benefit for this pipeline.

4.6 Alternative OpenMP designs

Table 8 compares each ablation with the base design on the stage it changes (5 test images). Dynamic scheduling of the 64 CLAHE tiles makes no measurable difference

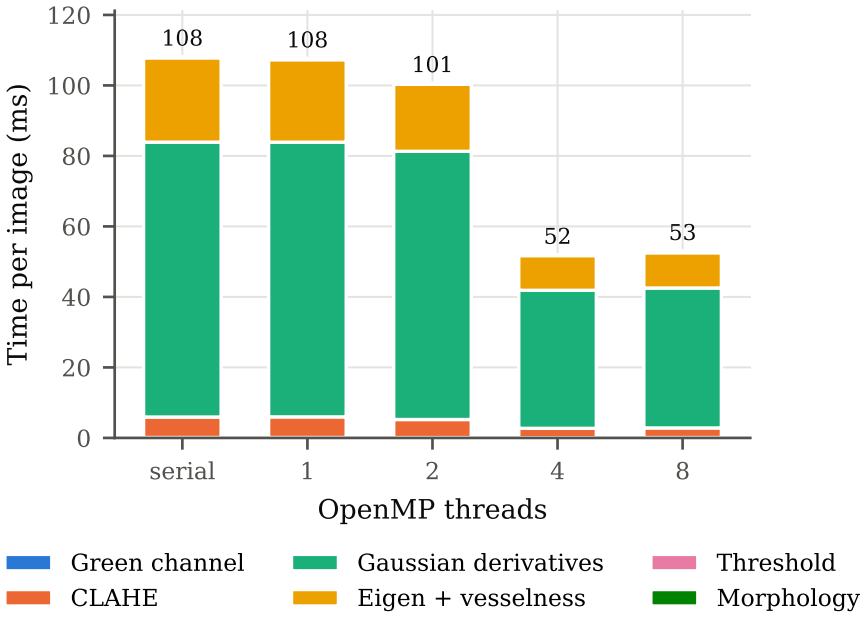


Fig. 2. Mean time per DRIVE test image by stage for the serial build and for OpenMP with 1–8 threads.

Table 7. Effect of thread placement on whole-pipeline speedup (5 DRIVE test images).

Placement	$S(2)$	$S(4)$	$S(8)$
close, `threads`	1.08	2.12	2.06
spread, `cores`	1.98	2.04	2.03
no binding	1.98	2.12	1.98

(ratios 0.99–1.00). Building the histograms pixel-parallel with atomic increments makes CLAHE 1.53× slower even on one thread, where the atomics cost time without any contention, and 1.68–1.70× slower at $p = 4$–8. Running the four Frangi scales concurrently matches the base design at $p \leq 2$ but is 1.17× slower at $p = 4$ and 1.21× at $p = 8$.

4.7 Resolution scaling

Table 9 and Fig. 6 show how the speedup changes with image size. Serial time grows from 108.9 ms at 0.33 megapixels to 8.67 s at 21.1 megapixels. The serial cost per megapixel rises from 330 ms to 411 ms (24%) at the largest size. The 4-thread speedup stays between 2.13 and 2.31 across all four sizes, and the 2-thread close-bound speedup rises from 1.08 to 1.16 above native size.

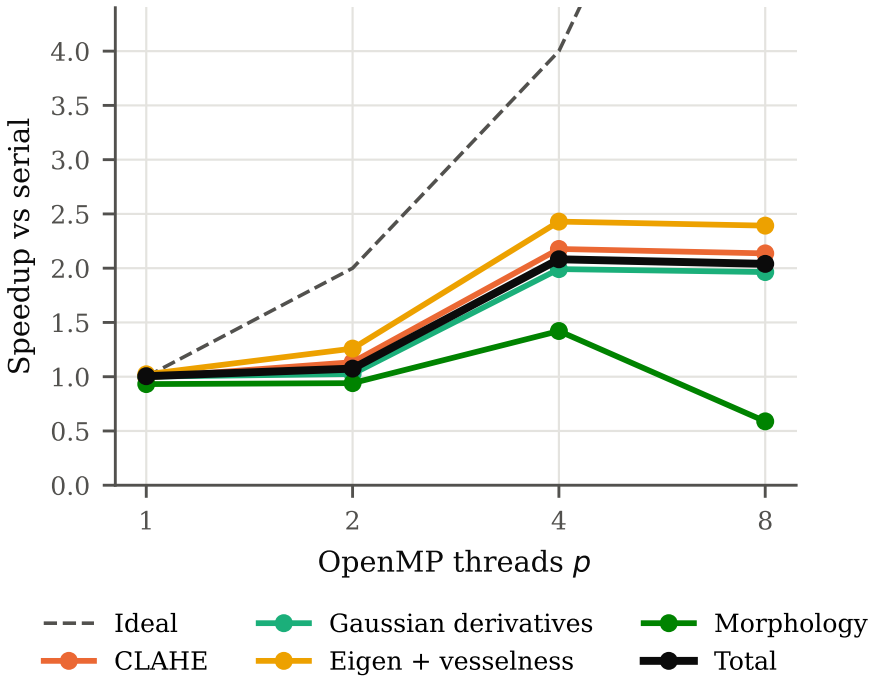


Fig. 3. Speedup over the serial build per stage and for the whole pipeline (close binding); the dashed line is ideal linear speedup.

4.8 Cross-dataset timing

On the 20 STARE images (700×605, 0.42 megapixels) the serial pipeline takes 138.6 ms per image, or 327 ms per megapixel, the same per-pixel cost as DRIVE. The speedups are 1.08, 2.09 and 2.05 at $p = 2$, 4 and 8, matching DRIVE within 0.01, so the parallel behaviour depends on image size rather than on image content.

5 Discussion

5.1 Hypotheses

H1 (sub-linear scaling, limited by physical cores): supported. The best speedup on two physical cores is 2.12, and almost all of it comes from the second core: two threads on two cores already reach 1.98, while the second hyperthread of each core adds only 7%. Hyper-Threading [12] helps little because the dominant Gaussian passes keep both hardware threads of a core waiting on the same load and floating-point units. With close binding the curve shows the topology directly: two threads on one core gain 7–8%, the same Hyper-Threading gain seen at $p = 4$. The Karp–Flatt fraction rises from 0.31 at

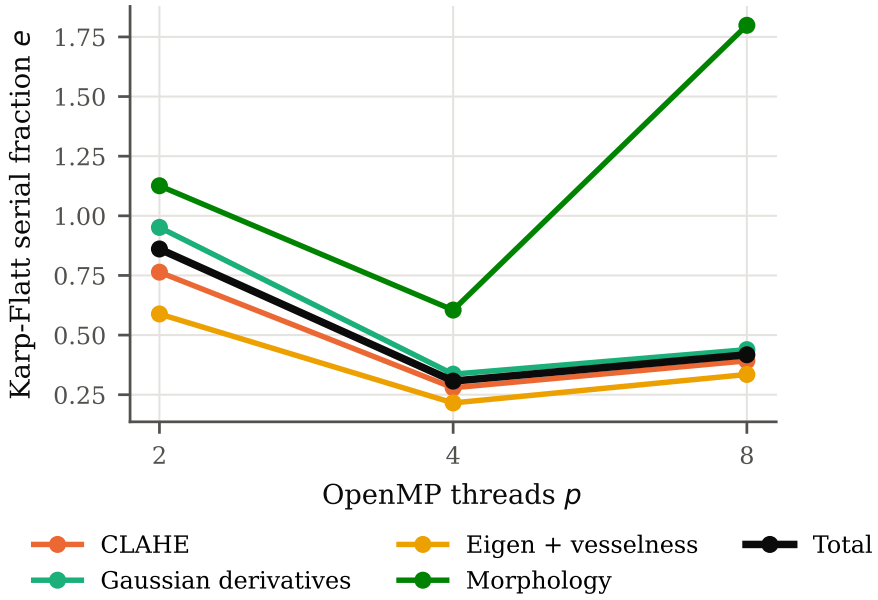


Fig. 4. Karp–Flatt serial fraction e per stage (close binding).

Table 8. Design ablations: stage time relative to the base design (ratio > 1 is slower).

Variant	Stage	$p = 1$	$p = 2$	$p = 4$	$p = 8$
dynamic tiles	CLAHE	0.99	1.00	0.99	1.00
atomic histogram	CLAHE	1.53	1.67	1.68	1.70
scale-parallel	Frangi	1.00	1.01	1.17	1.21

$p = 4$ to 0.42 at $p = 8$, which indicates growing overhead from oversubscription rather than a fixed serial part [10]. The high value at $p = 2$ (0.86) reflects the close placement, not serial code: with spread placement the efficiency at $p = 2$ is 0.99.

H2 (speedup stable or growing with image size): supported. The 4-thread speedup stays between 2.13 and 2.31 from 0.33 to 21.1 megapixels and is slightly higher above native size, where each parallel region has more work to amortise its fork and join. The serial cost per megapixel rises by 24% at 21.1 megapixels. At that size the six float images used per scale (24 bytes per pixel, about 500 MB) far exceed the 55 MiB L3, so the column passes stream from DRAM; the parallel version suffers the same effect, which is why the speedup does not change [21].

H3 (best decomposition differs by stage): supported. In CLAHE, one thread per tile needs no synchronisation, and the 64 tiles cost almost the same, so dynamic scheduling neither helps nor hurts. The atomic variant is 53% slower even on one thread, because every increment becomes a locked read-modify-write, and gets worse with more threads.

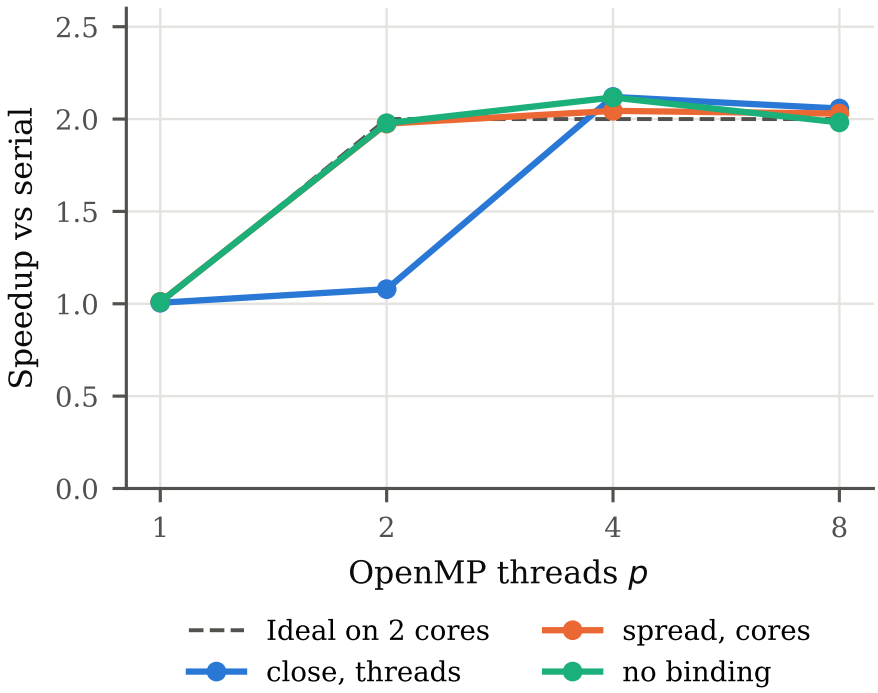


Fig. 5. Whole-pipeline speedup for three thread placements. The dashed line is the ideal for two physical cores.

In the Frangi stage the cost of a scale grows with its kernel length ($2\lceil 3\sigma \rceil + 1 = 7$, 13, 19 and 25 taps), so the thread that owns $\sigma = 4$ carries 39% of the convolution work, and running scales concurrently leaves threads idle once the short scales finish. The variants are equal up to $p = 2$ and fall behind only at $p \geq 4$.

H4 (accuracy below lightweight deep models): supported. The DRIVE test Dice of 0.7126 is below 0.8070 for LightVesselNet [16], 0.8328 for LFA-Net [14] and 0.8644 for LVS-Net [13]. The pipeline needs no annotated data, no weights and no deep-learning runtime. Its parameters transfer to STARE without retuning, losing 0.059 Dice, a drop driven mainly by images with bright lesions. The deep-model figures are taken from the authors' papers and protocols, so the comparison is indicative only.

5.2 Why the small stages scale badly

Morphology, thresholding and green-channel extraction each take well under a millisecond per image. A parallel region on this machine costs a few microseconds to fork and join, which is a large fraction of a 0.2 ms stage split four ways, and at $p = 8$ each logical CPU time-slices two threads, so a thread that is descheduled delays the whole region. Morphology runs four such regions back to back, each over only 0.33 MB of byte data,

Table 9. Resolution scaling (3 DRIVE test images, $p = 4$, close binding).

Scale	MP	Serial (ms)	OpenMP (ms)	S	MP/s
1×	0.33	108.9	51.1	2.13	6.45
2×	1.32	463.6	200.4	2.31	6.59
4×	5.28	1853.6	807.1	2.30	6.54
8×	21.12	8669.4	3804.5	2.28	5.55

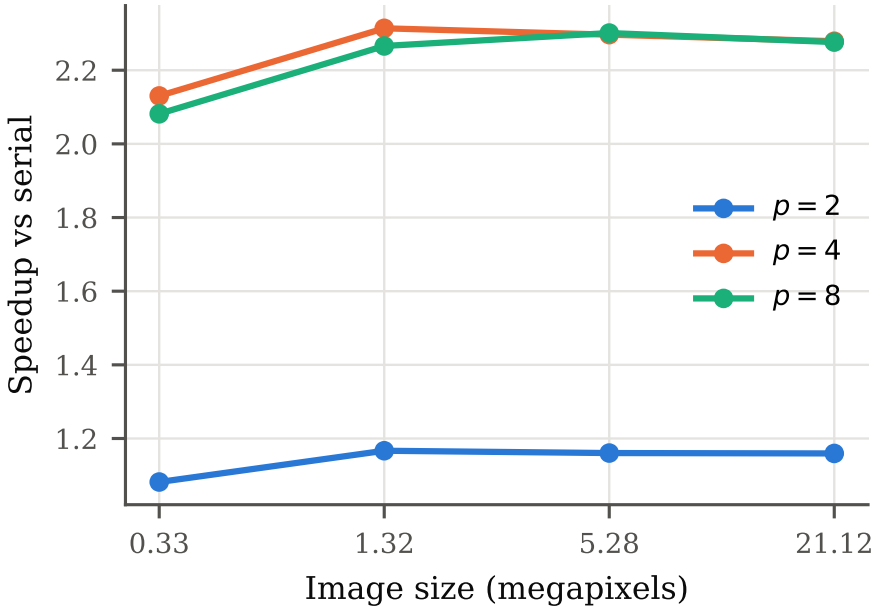


Fig. 6. Speedup over the serial build against image size for each thread count (close binding).

so it slows down at $p = 8$ (speedup 0.59). Because these stages make up under 0.5% of the time, their poor scaling has no visible effect on the total. Fusing them into one parallel region would be the fix if they ever mattered.

5.3 Practical implications

At 19.2 images per second on a 2-core VM, one such machine processes 10,000 fundus images in under nine minutes, so throughput is not the limiting factor for a screening camp at DRIVE resolution. At 21 megapixels it handles 0.26 images per second, where a second machine, more cores or a GPU would matter. Three practical lessons follow. First, parallelise the Gaussian-derivative stage first, since it is 72% of the time. Second, on small SMT machines, check thread placement before adding threads: close binding with fewer threads than logical CPUs can leave a whole core idle, while the default

unbound run happened to spread correctly. Third, prefer owner-computes histograms to atomics.

5.4 Limitations

The study uses one cloud VM with two physical cores shared with other tenants, so it says nothing about larger core counts, NUMA systems or dedicated hardware, and its timings carry more noise than a dedicated machine's; the coefficient of variation of the serial total was 2.6%. The CPU model is reported only as an Intel Xeon at 2.20 GHz (family 6, model 79). The placement and ablation experiments use five of the twenty test images. GPU implementations, which the original proposal planned, are outside the scope of this paper. The accuracy pipeline uses one global threshold and a fixed FOV erosion, and the STARE FOV masks are computed by thresholding rather than supplied with the dataset.

6 Conclusion

We parallelised a complete CLAHE–Frangi–morphology retinal vessel pipeline with OpenMP and measured it stage by stage against a pragma-free serial build of the same C code. The parallel version is bit-identical to the serial one and reaches 2.08× on two physical cores with Hyper-Threading, or 19.2 DRIVE images per second. Its accuracy (Dice 0.713 on DRIVE and 0.654 on STARE) is below that of lightweight deep models, but needs no training. The Gaussian-derivative passes dominate the cost. The design decisions that matter most are thread placement on SMT cores, owner-computes histograms, and parallelism inside each Frangi scale. Future work covers machines with more cores, cache blocking of the column passes, fusing the small stages into one parallel region, and the GPU comparison planned in the original proposal.

Code and data availability

The C source, the Kaggle driver, all CSV logs, the analysis scripts and a Colab demonstration notebook are in the project repository. DRIVE and STARE are available from their original providers [18,8].

Use of AI tools

Code, experiment scripts and the manuscript draft were produced with the assistance of an AI coding assistant (Claude, Anthropic) under the authors' direction. All numbers come from the authors' runs on the Kaggle VM, and the authors reviewed and take responsibility for the content.

References

1. Argüello, F., Vilariño, D.L., Heras, D.B., Nieto, A.: GPU-based segmentation of retinal blood vessels. Journal of Real-Time Image Processing **14**(4), 773–782 (2018). https://doi.org/10.1007/s11554-014-0469-z
2. Ben Sayadia, S., Elloumi, Y., Akil, M., Bedoui, M.H.: Efficient graphical-processor-unit parallelization algorithm for computing Eigen values. Journal of Electronic Imaging **29**(6), 063008 (2020). https://doi.org/10.1117/1.JEI.29.6.063008
3. Chen, C., Chuah, J.H., Ali, R., Wang, Y.: Retinal vessel segmentation using deep learning: A review. IEEE Access **9**, 111985–112004 (2021). https://doi.org/10.1109/ACCESS.2021.3102176
4. Frangi, A.F., Niessen, W.J., Vincken, K.L., Viergever, M.A.: Multiscale vessel enhancement filtering. In: Medical Image Computing and Computer-Assisted Intervention (MICCAI'98). Lecture Notes in Computer Science, vol. 1496, pp. 130–137. Springer, Berlin, Germany (1998)
5. Fraz, M.M., Remagnino, P., Hoppe, A., Uyyanonvara, B., Rudnicka, A.R., Owen, C.G., Barman, S.A.: Blood vessel segmentation methodologies in retinal images – a survey. Computer Methods and Programs in Biomedicine **108**(1), 407–433 (2012). https://doi.org/10.1016/j.cmpb.2012.03.009
6. Galdran, A., Anjos, A., Dolz, J., Chakor, H., Lombaert, H., Ben Ayed, I.: State-of-the-art retinal vessel segmentation with minimalistic models. Scientific Reports **12**, 6174 (2022). https://doi.org/10.1038/s41598-022-09675-y
7. Gojić, G., Kundačina, O., Mišković, D., Dragan, D.: Overview of deep learning methods for retinal vessel segmentation. arXiv:2306.06116 (2023)
8. Hoover, A., Kouznetsova, V., Goldbaum, M.: Locating blood vessels in retinal images by piecewise threshold probing of a matched filter response. IEEE Transactions on Medical Imaging **19**(3), 203–210 (2000)
9. International Diabetes Federation: IDF diabetes atlas, 11th edition. https://diabetesatlas.org (2025), india country profile: 89.8 million adults aged 20–79 with diabetes in 2024
10. Karp, A.H., Flatt, H.P.: Measuring parallel processor performance. Communications of the ACM **33**(5), 539–543 (1990)
11. Mahapatra, S., Agrawal, S., Mishro, P.K., Pachori, R.B.: A novel framework for retinal vessel segmentation using optimal improved Frangi filter and adaptive weighted spatial FCM. Computers in Biology and Medicine **147**, 105770 (2022). https://doi.org/10.1016/j.compbiomed.2022.105770
12. Marr, D.T., Binns, F., Hill, D.L., Hinton, G., Koufaty, D.A., Miller, J.A., Upton, M.: Hyper-threading technology architecture and microarchitecture. Intel Technology Journal **6**(1), 4–15 (2002)
13. Mehmood, M., Iqbal, S., Khan, T.M., Spence, I., Fahim, M.: LVS-Net: A lightweight vessels segmentation network for retinal image analysis. arXiv:2412.05968 (2024)
14. Mehmood, M., Spence, I., Fahim, M.: LFA-Net: A lightweight network with LiteFusion attention for retinal vessel segmentation. arXiv:2509.21738 (2025)
15. Nahar, S., Wang, Z., Arkfeld, E., McMahon, G., Hakozaki, H., Schöneberg, J.: GPU-accelerated MitoGraph for high-throughput three-dimensional mitochondrial morphology analysis. BMC Bioinformatics **27**, 206 (2026). https://doi.org/10.1186/s12859-026-06441-z
16. Sobhan, S., Jalil, F.: LightVesselNet: An ultra-lightweight sub-100K parameter network for retinal blood vessel segmentation. arXiv:2606.05354 (2026)

17. Song, Y., Li, C., Xiao, S., Xiao, H., Guo, B.: Unsharp masking image enhancement the parallel algorithm based on cross-platform. Scientific Reports **12**, 20175 (2022). https://doi.org/10.1038/s41598-022-21745-9
18. Staal, J., Abràmoff, M.D., Niemeijer, M., Viergever, M.A., van Ginneken, B.: Ridge-based vessel segmentation in color images of the retina. IEEE Transactions on Medical Imaging **23**(4), 501–509 (2004)
19. Sule, O.O.: A survey of deep learning for retinal blood vessel segmentation methods: Taxonomy, trends, challenges and future directions. IEEE Access **10**, 38202–38236 (2022). https://doi.org/10.1109/ACCESS.2022.3163247
20. van der Walt, S., Schönberger, J.L., Nunez-Iglesias, J., Boulogne, F., Warner, J.D., Yager, N., Gouillart, E., Yu, T., the scikit-image contributors: scikit-image: Image processing in Python. PeerJ **2**, e453 (2014). https://doi.org/10.7717/peerj.453
21. Williams, S., Waterman, A., Patterson, D.: Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM **52**(4), 65–76 (2009). https://doi.org/10.1145/1498765.1498785
22. Zuiderveld, K.: Contrast limited adaptive histogram equalization. In: Heckbert, P.S. (ed.) Graphics Gems IV, pp. 474–485. Academic Press, San Diego, CA, USA (1994)